

# NeuronFabric: A Fixed-Budget Hardware Architecture for On-Chip Transformer Training via Native Backpropagation

Architecture Design, Software Prototype, and the Vocabulary-Budget Constraint

Evgeny Ukladchikov  
Independent Research  
euklad@gmail.com

Code: [github.com/Binoculars-X/neuro-fabric](https://github.com/Binoculars-X/neuro-fabric)

May 2026

## Abstract

This paper describes **NeuronFabric**, a hardware architecture for transformer neural networks in which weights live permanently in local SRAM next to each compute unit, and both the forward pass and the backpropagation pass execute on that same hardware. The motivation is simple: every neural accelerator we are aware of either does inference only (Groq LPU, Apple ANE, IBM NorthPole) or offloads the weight update to a host CPU or external GPU cluster (Cerebras WSE-3, Tenstorrent Wormhole). NeuronFabric is an attempt to close that gap.

We report results from a complete C# software prototype covering **NeuronCore**, **AttentionCore** (Q/K/V projections, scaled dot-product attention, and backprop), **AttentionLayer** (multi-head with feedforward block), and **TransformerBus** (a full autoregressive language model). The prototype reaches 97%+ on MNIST and 99%+ on IRIS as basic correctness checks, then trains transformer language models on real dialogue corpora.

During these experiments we ran into a constraint that is not obvious from the literature: on a 100K-parameter chip, the embedding table dominates the parameter count and the transformer layers have almost nothing left to work with unless the vocabulary is kept small. We call this the **vocabulary-budget constraint**. Keeping vocabulary below roughly 150 tokens, we achieved 78.9% accuracy (cross-entropy loss 0.42) on a 49-token appointment scheduling domain; with a 302-token vocabulary on MultiWOZ 2.2 the same model reaches 50.8% accuracy after 100K training steps.

We also sketch a shared-embedding Mixture-of-Experts topology designed to separate the vocabulary cost from expert-chip capacity in a multi-chip pipeline, so that inter-chip traffic stays proportional to layer width regardless of how many chips are in the chain. FPGA implementation and a first-silicon tapeout via Google OpenMPW (SKY130, 130nm) are next on the roadmap.

Code: <https://github.com/Binoculars-X/neuro-fabric>

## 1 Introduction

The standard split between training hardware and inference hardware is so entrenched that it is rarely questioned. GPUs handle both, but at data-centre scale and megawatt power budgets. Inference ASICs (Groq LPU, Apple ANE, IBM NorthPole) are efficient but hardwired for forward

passes only—there is no path to update a weight. Cerebras and Tenstorrent support training but rely on the host CPU or an external optimiser for the actual weight-update step; the update does not happen on the chip that computed the activations.

The consequences of this split are not just architectural inconveniences:

1. **Weight traffic.** Each inference requires loading all weights from HBM memory. For a 70B parameter model, this is  $\sim 140$  GB of memory traffic per forward pass. The H100 generates 14 GB per inference for a 7B model [7].
2. **Training power.** GPT-4 scale training consumes  $\sim 1$  MW continuously over months. No existing ASIC supports on-chip training at any power level.
3. **No continuous learning.** Inference-only chips cannot learn from new data without sending weights to an external GPU cluster, retraining, and reloading— a cycle taking minutes to hours even for fine-tuning.

NeuronFabric addresses all three by keeping weights *statically* in local SRAM adjacent to each compute unit. A forward pass reads weights from SRAM; a backward pass writes updated weights to the same SRAM. No external memory. No weight traffic across chip boundaries. Training and inference on the same hardware.

## 1.1 Key Claims

1. A transformer with native backpropagation can be implemented with weights permanently in local SRAM, with zero off-chip weight traffic during both forward and backward passes.
2. At a fixed hardware parameter budget ( $\sim 100$ K), vocabulary size is the dominant constraint on output quality—a result we demonstrate empirically across three domains.
3. A shared-embedding MoE topology eliminates the vocabulary tax from expert chips, enabling multi-chip scaling where inter-chip bandwidth grows with layer width only, not with parameter count.

## 2 Architecture

### 2.1 NeuronCore: The Fundamental Compute Unit

A **NeuronCore** models a single synapse group: it holds its own weight vector  $\mathbf{w} \in \mathbb{R}^n$  in local storage (SRAM on FPGA/ASIC, array in prototype), computes a dot product on the forward pass, and updates weights in-place on the backward pass.

**Forward pass:**

$$z = \mathbf{w}^\top \mathbf{x} + b, \quad y = \sigma(z) \tag{1}$$

where  $\sigma$  is ReLU, sigmoid, or linear depending on layer position.

**Backward pass:**

$$\delta = \frac{\partial L}{\partial z} = g \cdot \sigma'(z) \tag{2}$$

$$\mathbf{w} \leftarrow \mathbf{w} - \eta \cdot \delta \cdot \mathbf{x}, \quad b \leftarrow b - \eta \cdot \delta \tag{3}$$

where  $g$  is the incoming gradient. The weight update is a direct SRAM write— on FPGA this maps to a BRAM write on the **Backward** signal.

Weights are He-initialised:  $w_i \sim \mathcal{N}(0, \sqrt{2/n})$ .

## 2.2 NeuronLayer: Parallel Core Dispatch

A **NeuronLayer** holds  $k$  **NeuronCore** instances and broadcasts the bus signal to all cores simultaneously—modelling the FPGA **always** blocks that fire in parallel. The layer collects the scalar output from each core into a vector for the next layer.

## 2.3 AttentionCore: Single Attention Head

**AttentionCore** implements one attention head with local projection weights  $W_Q, W_K, W_V \in \mathbb{R}^{d \times d_h}$  and  $W_O \in \mathbb{R}^{d_h \times d}$  stored in SRAM.

**Forward pass:**

$$Q = XW_Q, \quad K = XW_K, \quad V = XW_V \in \mathbb{R}^{T \times d_h} \quad (4)$$

$$A = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_h}} + M\right) \in \mathbb{R}^{T \times T} \quad (5)$$

$$\text{out} = AVW_O \in \mathbb{R}^{T \times d} \quad (6)$$

where  $M$  is the causal mask ( $-\infty$  for future positions). The softmax  $\exp(\cdot)$  is planned as a 256-entry LUT in BRAM for FPGA implementation, covering the score range  $[-8, 0]$  after scaling.

**Backward pass:** Standard attention backpropagation [1]. All gradients are computed before any weight updates to avoid contaminating gradient computation with partially updated weights—a subtle correctness requirement we discovered and validated via numerical gradient checking.

$$\frac{\partial L}{\partial W_O} = \hat{A}^\top \cdot \frac{\partial L}{\partial \text{out}} \quad (7)$$

$$\frac{\partial L}{\partial A} = \frac{\partial L}{\partial \text{out}} \cdot W_O^\top \cdot V^\top \quad (8)$$

$$\frac{\partial L}{\partial X} = \frac{\partial L}{\partial Q} W_Q^\top + \frac{\partial L}{\partial K} W_K^\top + \frac{\partial L}{\partial V} W_V^\top \quad (9)$$

The softmax backward uses the Jacobian-vector product:  $\frac{\partial L}{\partial s_i} = a_i \left( g_i - \sum_j g_j a_j \right)$  where  $a_i$  are softmax outputs and  $g_i$  are incoming gradients.

## 2.4 AttentionLayer: Multi-Head + Feedforward

**AttentionLayer** composes  $H$  parallel **AttentionCore** heads with a two-layer feedforward block and Pre-LayerNorm residual connections:

$$\hat{x} = \text{LN}_1(x) \quad (10)$$

$$x' = x + \frac{1}{H} \sum_{h=1}^H \text{Head}_h(\hat{x}) \quad (11)$$

$$x'' = x' + W_2 \cdot \text{GeLU}(W_1 \cdot \text{LN}_2(x')) \quad (12)$$

Pre-LN (normalise before attention, not after) is used for training stability [2].

## 2.5 TransformerBus: Full Autoregressive Language Model

TransformerBus orchestrates the complete forward/backward pass:

$$\text{tokens} \xrightarrow{\text{Embed}} \xrightarrow{+PE} \xrightarrow{L \times \text{AttentionLayer}} \xrightarrow{\text{logits}} \text{loss}$$

**Weight tying:** The output projection reuses the embedding table transposed:  $\text{logits}[t, v] = \text{layerOut}[t] \cdot \text{embedding}[v]$ . This eliminates  $|V| \times d$  parameters from the output head at zero accuracy cost, a critical saving at the 100K parameter budget.

**Bus signal protocol:** The architecture uses a 3-bit bus signal enum: `Idle` / `Encode` / `Forward` / `Backward` / `WeightRead` / `WeightWrite`, directly mapping to the FPGA control bus that drives all compute blocks.

## 2.6 Hardware Mapping

The software architecture is designed with a direct FPGA/ASIC mapping in mind:

| Software abstraction       | Hardware target                     |
|----------------------------|-------------------------------------|
| NeuronCore weight array    | BRAM block                          |
| NeuronCore.Forward()       | DSP48 chain                         |
| NeuronCore.Backward()      | BRAM write                          |
| NeuronLayer dispatch       | Parallel <code>always</code> blocks |
| AttentionCore              | DSP attention block                 |
| Softmax <code>exp()</code> | 256-entry LUT in BRAM               |
| TransformerBus             | SPI bus controller                  |

Table 1: Software-to-hardware mapping

## 3 The Vocabulary-Budget Constraint

When we started training on real corpora, the results were worse than expected— not because the backprop was wrong (the gradient checks passed), but because there was almost no model left after the embedding table took its share. The embedding table consumes  $|V| \times d$  parameters, and with weight tying the output projection costs nothing extra, but the embedding itself is unavoidable.

**Definition (Vocabulary Tax).** For a model with total parameter budget  $P$ , embedding dimension  $d$ , and vocabulary size  $|V|$ , the reasoning parameter budget is:

$$P_{\text{reason}} = P - |V| \cdot d \quad (13)$$

At  $P = 100\text{K}$  and  $d = 64$ :

| Vocabulary $ V $   | Embedding params | Reasoning params |
|--------------------|------------------|------------------|
| 1501 (TinyStories) | 96K              | <b>4K</b>        |
| 302 (MultiWOZ)     | 19.3K            | <b>80.7K</b>     |
| 49 (Appointment)   | 3.1K             | <b>96.9K</b>     |

When  $P_{\text{reason}}$  falls below  $\sim 20\text{K}$ , the transformer layers have insufficient capacity to form coherent patterns, producing recognisable words in incoherent order. We validate this empirically in Section 4.

**Implication for hardware design:** A fixed-budget chip must either (1) specialise to a constrained domain with small vocabulary, or (2) move the embedding to a separate chip that pays the vocabulary tax once for the entire multi-chip pipeline.

## 4 Experiments

All experiments use **TransformerBus** with the following base configuration unless noted: sequence length  $T = 32$ , embedding dim  $d = 64$ , 2 attention heads ( $d_h = 32$  each), feedforward dim  $f = 128$ , 2 transformer layers, vocabulary 256. Training uses SGD with gradient accumulation, batch size 12 (matched to physical core count for CPU efficiency).

### 4.1 MLP Baseline (NeuralBus)

| Dataset | Architecture  | Accuracy |
|---------|---------------|----------|
| MNIST   | 784–128–64–10 | 97.2%    |
| IRIS    | 4–16–8–3      | 99.3%    |

These results validate the correctness of **NeuronCore** forward/backward pipeline including He initialisation, ReLU, sigmoid, and softmax activations.

### 4.2 Transformer: Shakespeare (Byte-Level)

Training on the Shakespeare corpus (1M characters, byte-level vocab=256) with 115K parameters, the model plateaued at 30–33% accuracy after 40K steps. The embedding table at byte level costs only  $256 \times 64 = 16.4\text{K}$  parameters, leaving roughly 98.6K for the transformer layers—so the vocabulary-budget constraint is not the binding factor here.

After some investigation the more likely culprit is context length. Shakespeare byte-level prediction is a hard task; dependencies span dozens to hundreds of characters, and our fixed context window of  $T = 32$  tokens is too short to capture them. The architecture is functioning correctly—the gradient checks confirm that—but this task simply requires longer context than the prototype supports. We treated this as a task-selection lesson and moved to word-level corpora with stronger local structure.

### 4.3 Transformer: Appointment Domain (vocab=49)

A GPT-4-generated corpus of 600 dialogue lines across 4 appointment scheduling patterns, constrained to  $\leq 120$  unique words. The loader auto-extracts vocabulary by frequency, yielding vocab=49.

**Configuration:** 6 transformer layers,  $f = 256$ , batch=12, 20K steps.

| Steps | Eval Loss   | Accuracy     |
|-------|-------------|--------------|
| 5K    | 1.82        | 41.3%        |
| 10K   | 0.89        | 63.7%        |
| 20K   | <b>0.42</b> | <b>78.9%</b> |

**Perplexity at convergence:**  $e^{0.42} = 1.52$ .

**Sample outputs** (temperature=0.8):

- Prompt: *“can i book an appointment”*  
Output: *“is rescheduled for monday morning. please confirm if friday afternoon would suit you. we will send you a reminder.”*
- Prompt: *“sorry the doctor is not available”*  
Output: *“on monday morning. please confirm if thursday afternoon would suit you.”*

This demonstrates fluent, contextually coherent output from a 100K parameter model. The key enabler: with vocab=49, the embedding consumes only 3.1K parameters, leaving 96.9K for transformer reasoning capacity (Equation 13).

#### 4.4 Transformer: MultiWOZ 2.2 (vocab=302)

MultiWOZ 2.2 [4] is a multi-domain task-oriented dialogue dataset with 56,776 system turns and 905,979 tokens. We extract the top-300 most frequent words plus <UNK> and <EOS>, yielding vocab=302.

**Configuration:** same base configuration, 100K steps total, resumable checkpoint.

| Steps | Eval Loss   | Accuracy     | $\Delta$ Acc / 25K steps |
|-------|-------------|--------------|--------------------------|
| 20K   | 2.96        | 36.7%        | —                        |
| 50K   | 2.45        | 43.8%        | +7.1%                    |
| 75K   | 2.15        | 46.9%        | +3.1%                    |
| 100K  | <b>2.05</b> | <b>50.8%</b> | +3.9%                    |

**Perplexity at 100K steps:**  $e^{2.05} = 7.78$ .

The learning curve shows a capacity ceiling: the improvement rate halved between the 75K and 100K step blocks, indicating the model has reached its capacity limit for vocab=302 at 100K parameters. This is consistent with Equation 13: vocab=302 leaves 80.7K reasoning params, sufficient for structural patterns but insufficient for full multi-domain dialogue coverage.

**Sample outputs** demonstrate strong structural dialogue patterns: *“would you like me to book it for you”, “your reference number”, “moderately priced and free wifi”, “is there anything else i can help you with”*.

#### 4.5 Vocabulary-Budget Constraint: Validation Summary

| Domain      | $ V $ | $P_{\text{reason}}$ | Loss        | Accuracy   |
|-------------|-------|---------------------|-------------|------------|
| TinyStories | 1501  | 4K                  | 2.90        | 37%        |
| MultiWOZ    | 302   | 81K                 | 2.05        | 51%        |
| Appointment | 49    | 97K                 | <b>0.42</b> | <b>79%</b> |

The relationship between  $P_{\text{reason}}$  and output quality is consistent across three quite different domains. The TinyStories result is particularly striking: the model recognises English words and produces valid tokens, but the sentences make no sense—exactly what you would expect when 4K parameters are trying to learn dialogue structure. Once  $P_{\text{reason}}$  crosses roughly 80K the model produces recognisable patterns; at 97K it generates fluent domain text. We take this as reasonable empirical support for the constraint, though more data points across vocabulary sizes would strengthen it.

## 5 Shared-Embedding MoE Topology

The vocabulary-budget constraint motivates a multi-chip topology where the embedding tax is paid *once* by a dedicated gating chip, and expert chips contain only transformer layers with no embedding overhead.

### 5.1 Architecture

| Component                      | Params      | Role                       |
|--------------------------------|-------------|----------------------------|
| Gating chip                    | 200K        | Embed + transformer + gate |
| Expert chip $i$ ( $i = 1..9$ ) | 100K        | Transformer layers only    |
| <b>Total</b>                   | <b>1.1M</b> |                            |
| Active per token               | $\sim 200K$ | Gating + one expert        |

Table 2: MoE multi-chip topology

The gating chip performs:

1. Shared embedding lookup:  $\mathbf{e} = E[t] \in \mathbb{R}^d$
2. Positional encoding:  $\mathbf{x}_0 = \mathbf{e} + \text{PE}(t)$
3. Own transformer stack:  $\mathbf{h} = \text{TransformerLayers}(\mathbf{x}_0)$
4. Gate routing:  $i^* = \arg \max \text{softmax}(\mathbf{h}W_g)$
5. Output projection (weight-tied):  $\text{logits} = \mathbf{h}'E^\top$

The selected expert chip  $i^*$  receives hidden state  $\mathbf{h}$  as input, runs its transformer layers, and returns the updated hidden state to the gating chip.

### 5.2 Inter-Chip Bandwidth

The critical property: inter-chip traffic is *only the hidden state vector*  $\mathbf{h} \in \mathbb{R}^{T \times d}$ :

$$\text{bytes per sample} = T \times d \times 4 = 32 \times 64 \times 4 = 8192 \text{ bytes} \quad (14)$$

This is **fixed regardless of model depth or total parameter count**. Adding more chips adds more parameters but does not increase inter-chip bandwidth—in direct contrast to all systolic-array architectures where inter-chip traffic grows with layer width.

For the 10-chip TinyStories configuration ( $T = 64$ ,  $d = 128$ ):  $64 \times 128 \times 4 = 32,768$  bytes (32 KB) per sample, vs. 14 GB per sample for an equivalent GPU inference.

### 5.3 Load Balance Loss

To prevent expert collapse (all tokens routing to one expert), we add:

$$L_{\text{total}} = L_{\text{task}} + \alpha \cdot L_{\text{balance}}, \quad L_{\text{balance}} = N \sum_{i=1}^N f_i \cdot p_i \quad (15)$$

where  $f_i$  is the fraction of tokens routed to expert  $i$  and  $p_i$  is the mean routing probability for expert  $i$ . We use  $\alpha = 0.01$  following Switch Transformer [5].

## 5.4 FPGA / ASIC Multi-Chip Mapping

Listing 1: Multi-chip pipeline

```
[Host: Gating chip (200K)]
|-- Embed token t
|-- Run transformer layers
|-- Gate: select expert i*
|-- Send h via SPI to chip i*
[Expert chip i* (100K)]
|-- Receive h from SPI
|-- Run transformer layers
|-- Return h' via SPI
[Gating chip: project h' to logits]
```

At inference:  $N - 1$  chips draw zero compute power (idle). Only the gating chip and one expert chip are active. Power scales as  $O(1)$ , not  $O(N)$ .

## 6 Related Work

**Inference-only ASICs.** Groq LPU [11], Apple Neural Engine [14], and IBM NorthPole [8] achieve excellent inference efficiency through SRAM-local weight storage and systolic array or dataflow architectures. None support on-chip gradient computation. NeuronFabric adds the backward pass to this class.

**Neuromorphic chips.** Intel Loihi 2 [9], BrainChip Akida, and SpiNNaker [10] support local weight updates via spike-timing-dependent plasticity (STDP)—a Hebbian rule that does not implement gradient descent. STDP cannot train transformer attention to GPT-level accuracy on standard benchmarks. NeuronFabric uses standard gradient-descent backpropagation, not STDP.

**Training accelerators.** Cerebras WSE-3 [12] and Tenstorrent Wormhole [13] support training but require gradient aggregation on the host CPU or an external optimizer step—the weight update does not complete on the chip that computed the activations. NeuronFabric completes the weight update in the same SRAM block that stores the weight, with no host involvement.

**Mixture of Experts.** Switch Transformer [5] and Mixtral [6] demonstrate that sparse MoE reduces compute per token while scaling total parameters. Our shared-embedding topology extends this principle to fixed-budget hardware by separating the vocabulary cost from expert capacity.

**Small language models.** TinyStories [3] demonstrated that 1M parameter models can produce coherent English text on a constrained domain. Our vocabulary-budget analysis explains *why* this works and provides the design constraint  $|V| \lesssim 150$  for 100K-parameter hardware chips.

## 7 Future Work: FPGA and First Silicon

### 7.1 Phase 2b: FPGA Implementation (Planned)

The `TransformerBus` will be ported to SystemVerilog targeting Xilinx Kintex-7 or Pynq-Z2. The key engineering tasks are:

- Fixed-point arithmetic: Q8.8 weights with Q16.8 gradient accumulators
- Softmax: 256-entry exp LUT in BRAM, covering score range  $[-8, 0]$



- BRAM port contention: replicate sequence BRAM (2 copies, one per head; 8 KB each)
- Timing closure: target 50 MHz for proof; optimise to 100 MHz later

Target benchmarks: training power  $< 10$  W for attention backprop on appointment domain; correctness verified by comparing FPGA weight updates against C# reference.

## 7.2 Phase 3: First Silicon via Google OpenMPW

Upon successful FPGA validation, we will submit a tapeout to the Google OpenMPW program (SKY130 130nm process, free shuttle). Target chip specifications:  $\sim 100$ K parameters, 2 attention heads ( $d_h = 32$ ),  $T = 32$ , 50–100 MHz clock, estimated 10–50 mW inference power. The chip will run appointment domain inference and backpropagation, with weights persisted to external SPI NOR Flash.

# 8 Discussion

## 8.1 What Is and Is Not Proven

To be direct about the current state: the power numbers in this paper are projections, not measurements. The claim that NeuronFabric can train a transformer at milliwatts rests on FPGA and silicon work that has not been done yet. What the software prototype does establish is that the architecture is mathematically correct—the gradient checks pass to within 1% against double-precision numerical gradients across all components—and that the vocabulary-budget constraint is real and measurable in software.

Power estimates ( $< 10$  W on FPGA, 10–50 mW on SKY130 silicon) are derived from published FPGA characterisation data [15] and process parameters. We have not found a reason they should be wrong, but they need measurement to be convincing.

## 8.2 Fixed-Point Convergence Risk

The principal empirical unknown for the FPGA implementation is gradient underflow under fixed-point arithmetic. Q8.8 (8 integer bits, 8 fractional bits) provides  $\sim 0.004$  precision for gradient values, which may be insufficient for attention softmax backprop. We plan to use Q12.4 or Q16.0 accumulator format for gradient registers, validated by comparing FPGA weight trajectories against the C# float reference.

## 8.3 Comparison to Inference Power

At 130nm (Phase 3 silicon), the NeuronFabric 100K-parameter chip is not competitive with 5nm inference chips on throughput or absolute performance. The claim is not performance parity—it is **training capability** (native backprop) at a power level ( $< 50$  mW) that is impossible for any existing training system.

# 9 Conclusion

NeuronFabric started as a question: can you build a chip that trains its own weights, without shipping them off-chip, at a power budget that does not require a data centre? This paper is the first

step toward answering that—a software prototype that proves the architecture is correct, plus some empirical findings about what actually limits a 100K-parameter model. The main contributions are:

1. A complete software prototype of the architecture (**NeuronCore**, **AttentionCore**, **AttentionLayer**, **TransformerBus**) with 61 passing unit, integration, and gradient-check tests.
2. Empirical validation of the **vocabulary-budget constraint**: at 100K parameters with  $d = 64$ , vocabulary  $\leq \sim 150$  tokens is required for coherent output. This result holds across three distinct domains and explains prior observations in the small-LM literature.
3. A **shared-embedding MoE topology** that eliminates embedding overhead from expert chips, enabling multi-chip scaling with inter-chip bandwidth proportional to layer width only.
4. Demonstration of fluent domain-specialist generation at 100K parameters: 78.9% accuracy (loss 0.42, perplexity 1.52) on appointment dialogue, 50.8% accuracy on MultiWOZ 2.2.

FPGA implementation and silicon tapeout are planned as future work. All code is open source at <https://github.com/Binoculars-X/neuro-fabric>.

## References

- [1] A. Vaswani et al., “Attention is all you need,” *NeurIPS*, 2017.
- [2] R. Xiong et al., “On layer normalization in the transformer architecture,” *ICML*, 2020.
- [3] R. Eldan and Y. Li, “TinyStories: How small can language models be and still speak coherent English?,” *arXiv:2305.07759*, 2023.
- [4] P. Budzianowski et al., “MultiWOZ – a large-scale multi-domain wizard-of-oz dataset for task-oriented dialogue modelling,” *EMNLP*, 2019.
- [5] W. Fedus, B. Zoph, and N. Shazeer, “Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity,” *JMLR*, 2022.
- [6] A. Q. Jiang et al., “Mixtral of experts,” *arXiv:2401.04088*, 2024.
- [7] NVIDIA Corporation, “H100 Tensor Core GPU Architecture,” Technical whitepaper, 2022.
- [8] D. S. Modha et al., “Neural inference at the frontier of energy, space, and time,” *Science*, 382(6671):329–335, 2023.
- [9] M. Davies et al., “Loihi: A neuromorphic manycore processor with on-chip learning,” *IEEE Micro*, 38(1):82–99, 2018.
- [10] M. Furber et al., “The SpiNNaker project,” *Proceedings of the IEEE*, 102(5):652–665, 2014.
- [11] J. Ross et al., “Groq: A fast, flexible AI chip,” Technical report, Groq Inc., 2022.
- [12] Cerebras Systems, “Cerebras Wafer-Scale Engine 3,” Product brief, 2023.
- [13] Tenstorrent Inc., “Wormhole Architecture,” Technical documentation, 2023.
- [14] Apple Inc., “Accelerating Machine Learning with Apple Silicon,” Developer documentation, 2022.
- [15] AMD/Xilinx, “7 Series FPGAs Data Sheet: DC and Switching Characteristics,” DS182, 2022.